**Adaptive Recipe Recommendation  AI Culinary Copilot**
**Système adaptatif de recommandation personnalisée de recettes et copilote culinaire**
Food.com – Recommandation, prédiction de préférences et RAG

## Contexte et problématique

Les plateformes culinaires proposent un très grand nombre de recettes, mais les recommandations classiques restent souvent basées sur des similarités générales et ne prennent pas suffisamment en compte le profil de chaque utilisateur, son historique d'interactions et ses contraintes au moment de la recherche.

Le projet consiste à concevoir un système intelligent capable de construire et d'adapter progressivement le profil d'un utilisateur afin de lui proposer des recettes personnalisées.

Le système combinera plusieurs sources d'information :

- les caractéristiques des recettes ;
- les ingrédients, catégories, tags et informations nutritionnelles ;
- les temps de préparation et de cuisson ;
- les interactions utilisateur–recette et les notes ;
- les préférences et contraintes exprimées par l'utilisateur.

L'objectif n'est donc pas uniquement de recommander des recettes similaires, mais de construire une chaîne complète allant de la génération de candidats jusqu'au classement personnalisé, puis d'assister l'utilisateur à l'aide d'un copilote culinaire basé sur un système RAG.

## Objectifs du projet

Le système devra permettre de :

- construire un profil utilisateur à partir de ses interactions et préférences ;
  “‘
- générer des recommandations à l'aide de plusieurs stratégies de Machine Learning ;
- combiner les approches Content-Based et Collaborative Filtering dans un système hybride ;
- prédire la probabilité qu'un utilisateur apprécie une recette donnée ;
- adapter le classement des recettes en fonction des préférences et contraintes actuelles de l'utilisateur ;
- gérer le problème de *cold-start* pour les nouveaux utilisateurs ;
- exploiter les recettes comme base documentaire pour un copilote culinaire utilisant le RAG ;
- comparer expérimentalement les différentes approches et analyser leurs limites. “‘

- Utilisation de représentations TF-IDF et/ou d'embeddings pour calculer la similarité entre le profil utilisateur et les recettes.
- **Collaborative Filtering :** exploitation des interactions utilisateur–recette et des notes historiques afin d'identifier des recettes appréciées par des utilisateurs ayant des comportements similaires.
- Comparaison de plusieurs méthodes de recommandation, par exemple une approche Content-Based, une approche Collaborative Filtering et différentes stratégies hybrides.
- **Candidate Generation :** génération d'un ensemble de recettes candidates avant l'étape de classement final.
- **Constraint-aware Recommendation :** prise en compte des contraintes exprimées par l'utilisateur : ingrédients disponibles, temps maximal, catégorie et critères nutritionnels disponibles dans les données.
- Production d'un classement Top-K personnalisé. "'

### Composant 2 — Modèle de préférence et ranking adaptatif

Ce composant aura pour objectif d'estimer la compatibilité entre un utilisateur et une recette candidate.

"'

- Formulation du problème comme une tâche de classification ou de prédiction de préférence.
- Construction de variables combinant :
  - caractéristiques de la recette ;
  - historique des interactions de l'utilisateur ;
  - statistiques sur les notes précédentes ;
  - fréquence d'interaction ;
  - similarité avec les recettes précédemment appréciées ;
  - caractéristiques communes entre le profil utilisateur et la recette candidate.
- Comparaison de plusieurs modèles : Logistic Regression, Random Forest, XGBoost et/ou LightGBM.
- Production d'un score de préférence pour chaque paire utilisateur–recette.
- Utilisation de ce score dans une étape de **re-ranking** afin d'améliorer le classement produit par le système hybride.
- Mise en place d'une stratégie de **cold-start** : un nouvel utilisateur peut fournir quelques préférences initiales via un questionnaire afin de construire un premier profil.
- **Adaptation progressive du profil :** les nouvelles interactions et évaluations de l'utilisateur peuvent être intégrées afin de faire évoluer ses préférences. "'

### Composant 3 — Copilote culinaire basé sur le RAG

Le troisième composant permettra à l'utilisateur d'interagir avec la base de recettes en langage naturel.

‘‘‘

- Construction d'une base documentaire à partir des recettes, ingrédients, descriptions, instructions, tags et informations disponibles.
- Génération d'embeddings à l'aide de Sentence-Transformers.
- Indexation vectorielle avec FAISS ou Chroma.
- Mise en place d'un pipeline RAG :
  1. compréhension de la requête ;
  2. recherche des recettes pertinentes ;
  3. récupération des informations utiles ;
  4. génération d'une réponse à partir du contexte récupéré.
- Utilisation d'un modèle de langage de petite taille tel que Phi-3.5-mini ou Gemma-2-2B, selon les ressources disponibles.
- Possibilité d'étudier une adaptation légère du modèle avec LoRA/QLoRA si les ressources et le temps le permettent.
- Le copilote pourra notamment répondre à des demandes telles que :
  - “Que puis-je préparer avec ces ingrédients ?”
  - “Quelles recettes correspondent à mes préférences ?”
  - “Quelle recette prend moins de 30 minutes ?”
  - “Comment adapter cette recette pour deux personnes ?”
  - “Quels ingrédients sont utilisés dans cette recette ?”
  - “Pourquoi cette recette m'a-t-elle été recommandée ?”
- Les réponses devront être basées autant que possible sur les informations récupérées depuis la base documentaire afin de limiter les réponses non supportées par les données. ‘‘‘

## Pipeline global

Le fonctionnement global du système peut être résumé comme suit :

**Données Food.com → Prétraitement → Profil utilisateur →**
**Génération de candidats → Recommandation hybride →**
**Prédiction de préférence → Re-ranking → Top-K personnalisé**

En parallèle, le copilote exploite la même base de recettes :

**Requête utilisateur → Embedding → Recherche vectorielle →**
**Contexte pertinent → SLM → Réponse contextualisée**

## Approches et modèles étudiés

| **Composant** | **Approches étudiées** | **Objectif** |
|---|---|---|
| Recommandation | Content-Based : TF-IDF / embeddings + similarité<br>Collaborative Filtering : Matrix Factorization / SVD<br>Approches hybrides | Comparer différentes stratégies de génération et de classement |
| Prédiction de préférence | Logistic Regression, Random Forest, XGBoost / LightGBM | Estimer la compatibilité utilisateur–recette et améliorer le ranking |
| Copilote RAG | Sentence-Transformers + FAISS/Chroma + SLM | Répondre aux requêtes à partir des recettes disponibles |
| Adaptation utilisateur | Profil initial + historique d'interactions + mise à jour progressive | Gérer le cold-start et l'évolution des préférences |

## Évaluation expérimentale

Une partie importante du projet sera consacrée à l'évaluation et à la comparaison des approches.

### Évaluation du système de recommandation

Les performances seront évaluées avec des métriques adaptées aux systèmes de recommandation :

- Precision@K ;
- Recall@K ;
- MAP@K ;
- NDCG@K ;
- MRR.

Les différentes stratégies seront comparées dans les mêmes conditions expérimentales afin d'étudier l'apport du modèle hybride et du re-ranking.

### Évaluation du modèle de préférence

Selon la formulation retenue, les métriques pourront inclure :

- Precision ;
- Recall ;
- F1-score ;
- ROC-AUC ;
- Accuracy.

### Évaluation du RAG

Le système RAG sera évalué sur plusieurs dimensions :

- pertinence des documents récupérés ;
- qualité du contexte transmis au modèle ;
- fidélité de la réponse aux données récupérées ;
- taux de réponses contenant des informations non supportées ;
- qualité des réponses sur un ensemble de requêtes de test.

## Défis techniques

Le projet permettra notamment d'étudier les problèmes suivants :

- **Sparsité des interactions :** certains utilisateurs évaluent peu de recettes, ce qui limite les informations disponibles pour personnaliser leurs recommandations.
- **Cold-start :** recommandation pour les nouveaux utilisateurs disposant de peu ou aucune interaction.
- **Qualité des données :** valeurs manquantes, données textuelles hétérogènes, informations nutritionnelles incomplètes et différences dans les interactions.
- **Biais et déséquilibre des interactions :** certaines recettes peuvent être beaucoup plus représentées que d'autres dans les interactions.
- **Temporalité :** séparation correcte des données d'entraînement et de test afin de limiter les risques de fuite d'information lors de l'évaluation.
- **Personnalisation dynamique :** adaptation du profil utilisateur lorsque de nouvelles interactions deviennent disponibles.
- **Re-ranking :** combinaison de plusieurs scores provenant des différents composants du système.
- **Grounding du RAG :** limitation des réponses qui ne peuvent pas être justifiées par les informations récupérées depuis les recettes.

## Gestion expérimentale et MLOps

Afin de garantir la reproductibilité du projet, une chaîne MLOps sera mise en place.

- **DVC :** versionnement des données et des différentes étapes de préparation.
- **MLflow :** suivi des expériences, hyperparamètres, métriques et modèles.
- **Git / GitHub :** gestion du code source et collaboration entre les membres du groupe.
- **API :** exposition des modèles et du système de recommandation via FastAPI ou Flask.

- **Interface web :** interface légère permettant de saisir les préférences et contraintes, consulter les recommandations et interagir avec le copilote.
- **Reproductibilité :** organisation du pipeline afin de pouvoir reproduire les principales expérimentations et résultats.

## Livrables

- **Pipeline de données :** nettoyage, transformation et préparation des recettes et interactions, versionnés avec DVC.
- **Benchmark de recommandation :** comparaison Content-Based, Collaborative Filtering et Hybrid avec analyse des métriques Top-K.
- **Modèle de préférence :** modèle supervisé permettant d'estimer la compatibilité utilisateur–recette.
- **Système de re-ranking adaptatif :** combinaison des scores de recommandation et de préférence afin de produire un classement personnalisé.
- **Stratégie cold-start :** mécanisme permettant de générer un premier profil utilisateur à partir de préférences initiales.
- **Copilote culinaire RAG :** système permettant d'interroger les recettes en langage naturel.
- **Application web :** interface regroupant profil utilisateur, contraintes, recommandations et copilote.
- **Suivi expérimental MLflow :** expériences, métriques et modèles enregistrés.
- **Rapport expérimental :** comparaison des approches, analyse des erreurs, étude du cold-start, limites et perspectives.

## Données

Le projet s'appuiera sur un dataset public basé sur les données de **Food.com**, contenant deux principales catégories d'informations.

### Recettes

Les données de recettes comprennent notamment :

- identifiant de recette ;
- nom ;
- ingrédients ;
- tags ;
- description ;
- étapes de préparation ;
- temps de préparation et/ou de cuisson ;
- informations nutritionnelles ;
- nombre d'étapes et d'ingrédients.

### Interactions utilisateurs

Les données d'interaction comprennent notamment :

- identifiant utilisateur ;
- identifiant recette ;
- note attribuée ;
- texte de l'avis ;
- date de l'interaction.

Ces deux ensembles permettent de construire à la fois les modèles de recommandation, les modèles de préférence et la base documentaire utilisée par le copilote.

Les étapes de préparation permettront également d'étudier des problèmes réalistes de qualité des données tels que les valeurs manquantes, les formats hétérogènes et la forte variation du nombre d'interactions entre utilisateurs.

## Répartition proposée pour trois membres

Le projet est structuré en trois axes techniques complémentaires afin de permettre un travail parallèle entre les membres du groupe.

| **Membre** | **Responsabilité principale** | **Travail** |
|---|---|---|
| Membre 1 | Système de recommandation | Prétraitement des interactions, Content-Based, Collaborative Filtering, Hybrid, génération de candidats, évaluation Top-K. |
| Membre 2 | Préférence et ranking adaptatif | Feature engineering utilisateur–recette, modèles supervisés, cold-start, score de préférence, re-ranking et analyse des erreurs. |
| Membre 3 | Copilote RAG | Préparation de la base documentaire, embeddings, FAISS/Chroma, pipeline RAG, SLM, évaluation du grounding et intégration avec l'API. |
| **Travail commun : API, interface, DVC, MLflow, Git, intégration et tests** | | |

## Technologies envisagées

- **Data / ML :** Python, Pandas, NumPy, Scikit-learn
- **Modèles :** PyTorch, XGBoost, LightGBM
- **NLP / Embeddings :** Sentence-Transformers
- **Recherche vectorielle :** FAISS ou Chroma
- **LLM / RAG :** Phi-3.5-mini ou Gemma-2-2B, LangChain

- **MLOps :** MLflow, DVC
- **Versionnement :** Git / GitHub
- **Environnement :** Google Colab / Kaggle
- **Backend :** FastAPI ou Flask
- **Frontend :** React ou interface web légère
- **Déploiement :** Docker, si le temps le permet

## Résultat attendu

Le résultat final sera un prototype complet de plateforme culinaire intelligente combinant :

**Recommandation personnalisée + Prédiction de préférences + Adaptation dynamique + Gestion des contraintes + Copilote RAG + MLOps**

L'intérêt du projet réside dans l'intégration de plusieurs techniques de Data Science, Machine Learning, recommandation, NLP et GenAI au sein d'un même système, avec une évaluation expérimentale permettant de mesurer l'apport de chaque composant.